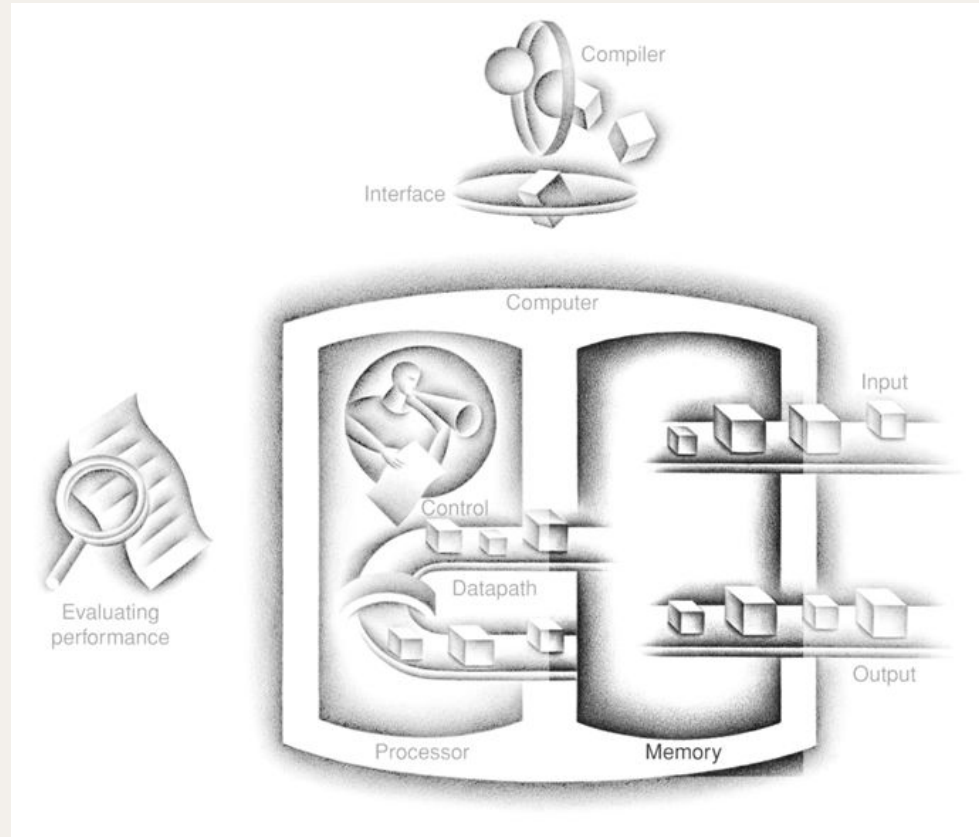


Sistema de Memória

João Canas Ferreira / António José Araújo
Fevereiro de 2025

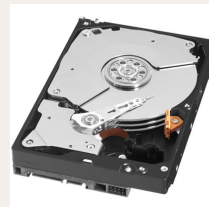
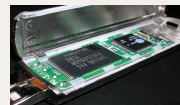
Organização de um computador: memória



Memória externa

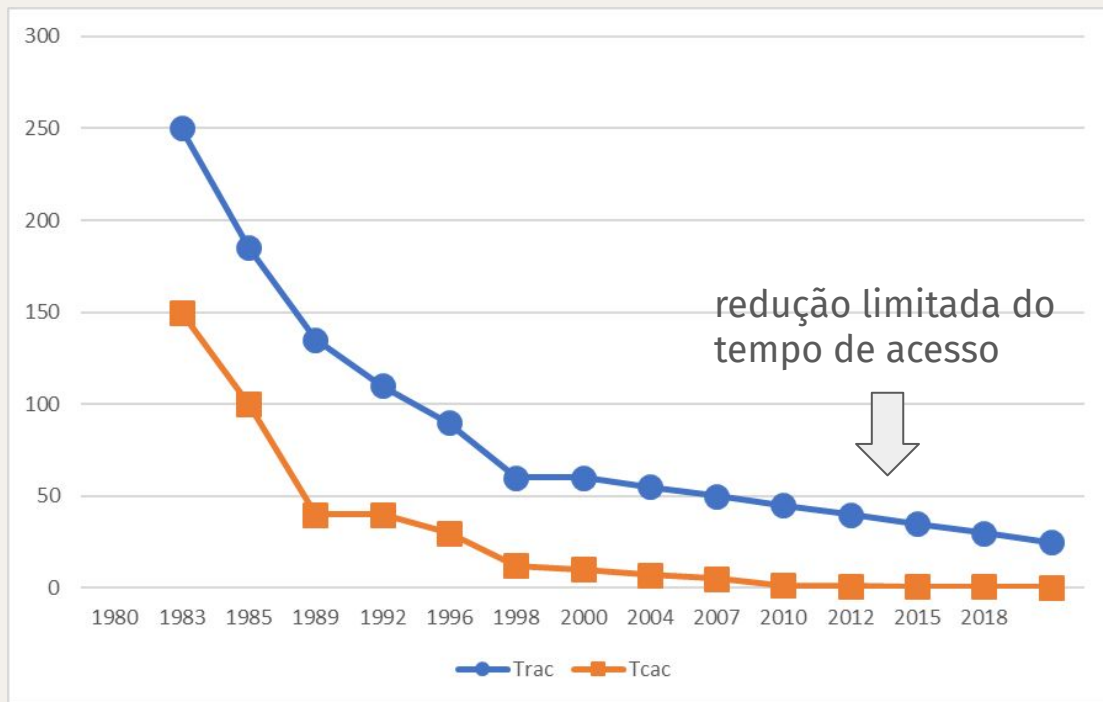
Tecnologias de memória

- Memória estática (Static RAM — SRAM) memória volátil
 - Tempo de acesso: 0,5 ns – 25 ns
 - Preço: 500 € – 1000 € por GB
- Memória dinâmica (Dynamic RAM — DRAM) memória volátil
 - Tempo de acesso: 50 ns – 70 ns
 - Preço: 3 € – 6 € por GB
- Memória Flash (não volátil)
 - Tempo de acesso: 5 μ s – 50 μ s
 - Preço: Preço: 0,06 € – 0,12 € por GB
- Disco magnético (não volátil)
 - Tempo de acesso: 5 ms – 20 ms
 - Preço: 0,01 € – 0,02 € por GB



Evolução dos circuitos integrados DRAM

Ano	Capacidade (bits)	Custo (€/GiB)
1980	64 Kib	6480000
1983	256 Kib	1980000
1985	1 Mib	720000
1989	4 Mib	128000
1992	16 Mib	30000
1996	64 Mib	9000
1998	128 Mib	900
2000	256 Mib	840
2004	512 Mib	150
2007	1 Gib	40
2010	2 Gib	13
2012	4 Gib	5
2015	8 Gib	7
2018	16 Gib	6



t_{RAC} : tempo de acesso para leitura (nova linha)

t_{CAC} : tempo de acesso para leitura (mesma linha)

Prefixos binários (revisão)

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%
ronnabyte	RB	10^{27}	robibyte	RiB	2^{90}	24%
queccabyte	QB	10^{30}	quebibyte	QiB	2^{100}	27%

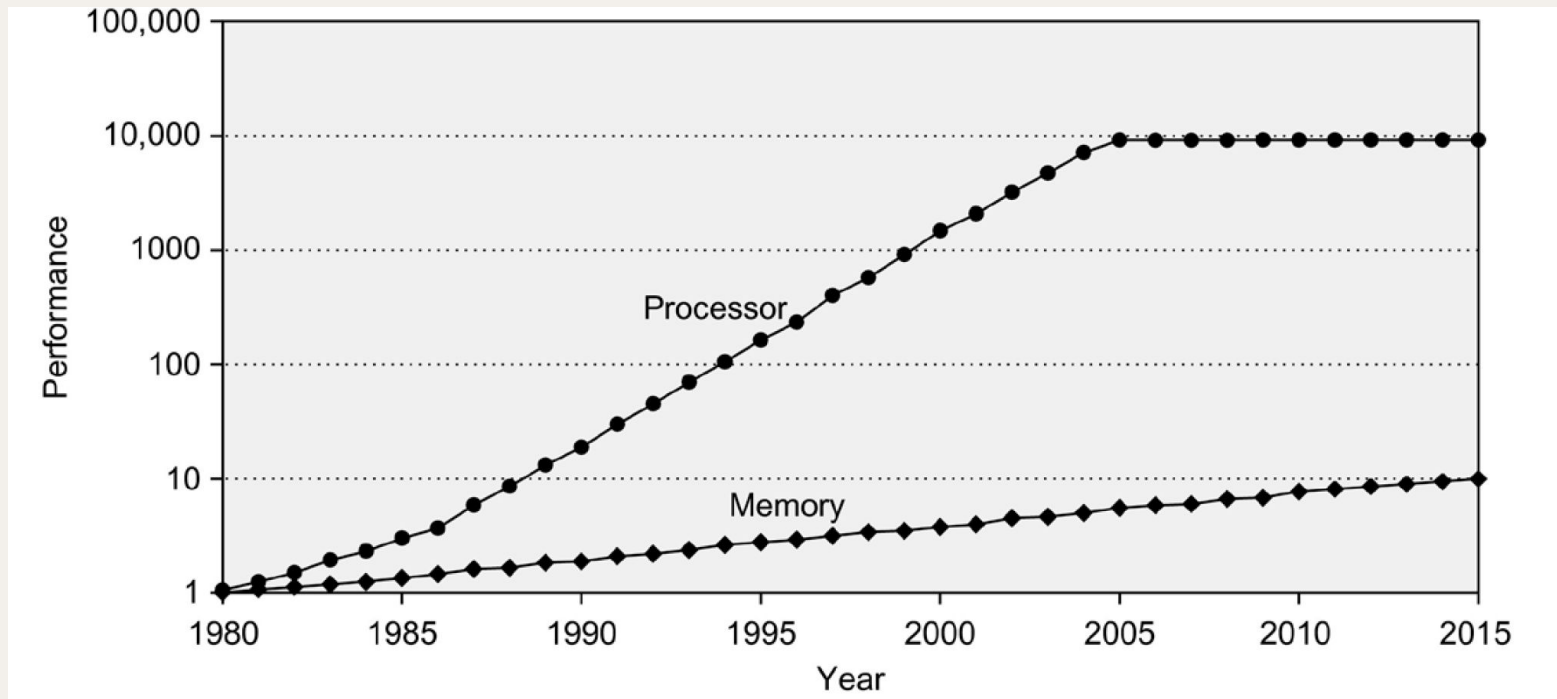
Organização geral do sistema de memória

Frequência de relógio vs. tempo de acesso

- O tempo de acesso a memórias DRAM é (relativamente) **muito elevado**.
- Exemplo: CPU de 1 GHz e CPI=1
 - Obter uma instrução a cada nanossegundo (10^9 Hz $\rightarrow$ período: 10^{-9} s = 1 ns)
 - Acesso a DRAM necessita de 50 ns
 - **Equivale a 50 ciclos de relógio do CPU**
 - O CPU passaria a maioria do tempo à espera de instruções / dados de memória
- Alternativa: usar SRAM.
 - Apenas é (economicamente) possível ter quantidade limitada de memória SRAM.

Como resolver este problema?

Tempo de acesso a memória vs. período de relógio (CPU)



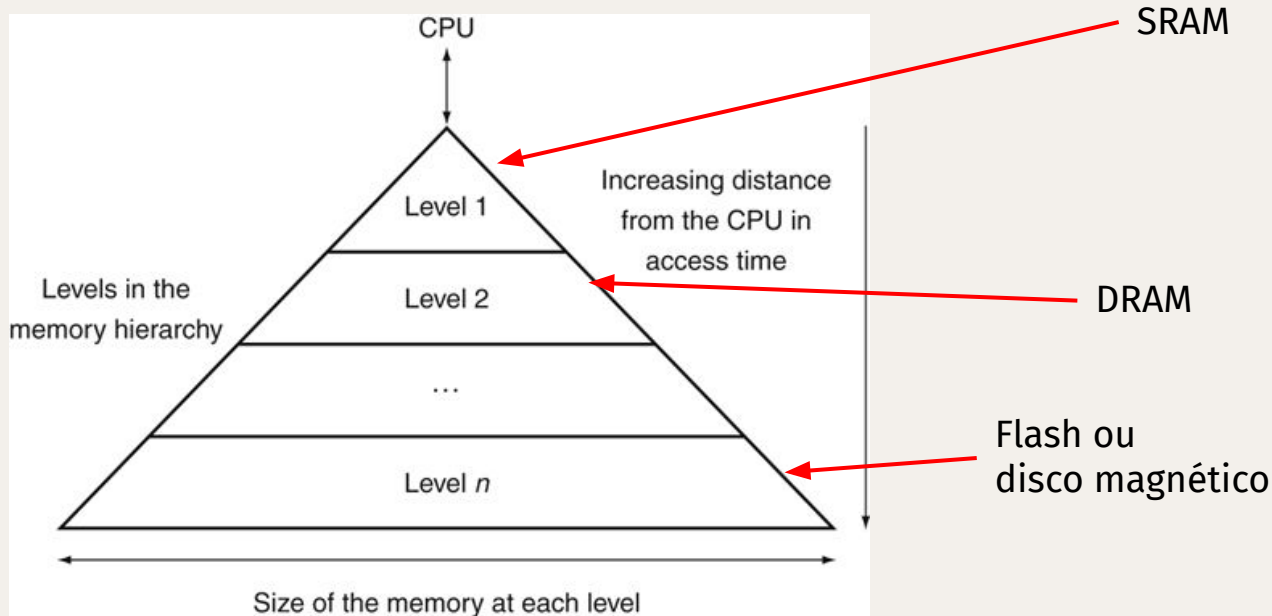
Tempo entre pedidos de acessos a memória (feitos pelo CPU) e a latência de um acesso a DRAM. Base: 1980.

Níveis da hierarquia de memória

- Memória ideal

- Tempo de acesso de SRAM
- Preço de disco magnético

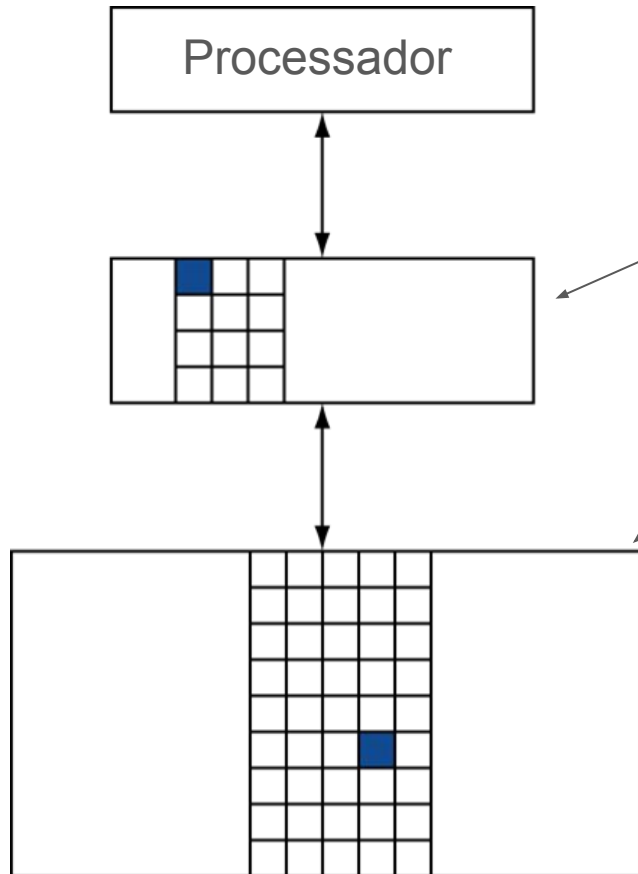
- Capacidade aumenta
- Custo baixa
- Rapidez baixa



Princípio da localidade para acessos à memória

- Os programas acedem a uma pequena proporção do seu espaço de endereços em qualquer altura.
 - “Localidade das referências”
- **Localidade temporal**
 - Posições accedidas recentemente são suscetíveis de serem novamente acedidas em breve.
 - Exemplo: instruções num ciclo, variáveis de indução
- **Localidade espacial**
 - É provável que posições próximas das que foram acedidas recentemente sejam acedidas em breve.
 - Exemplo: acesso (sequencial) a instruções, acesso a dados de uma sequência, acesso a variáveis locais

Aproveitamento da localidade



- Guardar instruções/dados em “uso” na memória SRAM.
- Guardar o programa e dados em memória DRAM.
- **Copiar informação entre níveis quando necessário.**
- Abordagem aplicável entre dois níveis adjacentes da hierarquia de memória.

Memória *Cache*

Memória cache

- Memória cache de nível 1
 - O nível da hierarquia de memória de nível mais próximo do CPU



1 a: *a hiding place especially for concealing and preserving provisions or implements; b:* *a secure place of storage*

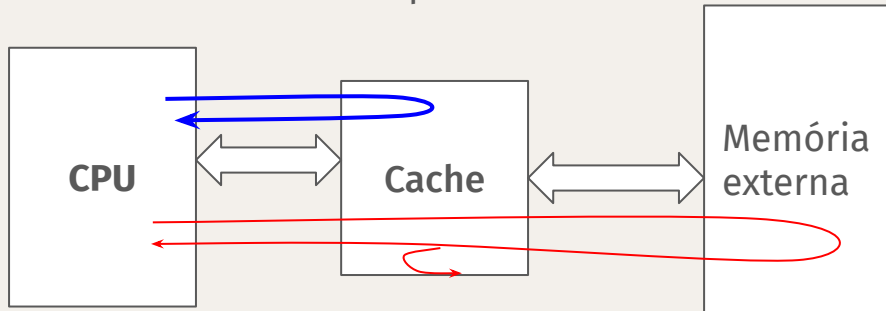
2: *something hidden or stored in a cache*

3: *a computer memory with very short access time used for storage of frequently or recently used instructions or data*

“cache”, Merriam-Webster.com Dictionary,
<https://www.merriam-webster.com/dictionary/cache>.
Accessed 19/03/2023.

Modelo conceitual de funcionamento (leitura)

1. CPU faz um pedido de acesso à memória
 - Indica o endereço
2. O pedido é processado pela memória cache
 - Se os dados correspondentes ao endereço estiverem em cache: transferir dados para CPU (e terminar).
 - Se os dados não estiverem em cache:
 - Obter os dados de memória externa (DRAM)
 - Guardar uma cópia dos dados na memória cache
 - Transferir os novos dados para CPU



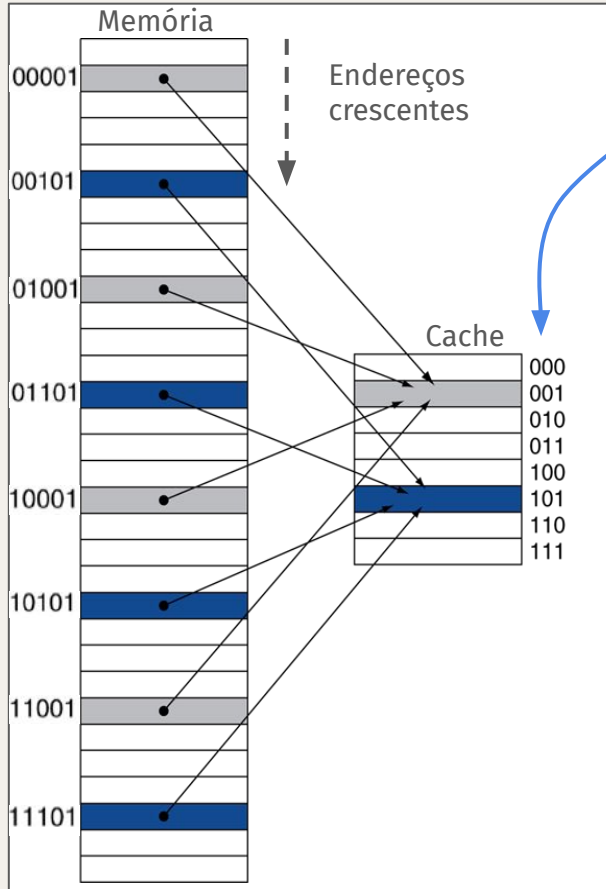
Dados em cache:
acerto ou *cache hit*

Dados não estão em cache:
falta ou *cache miss*

Questões importantes para funcionamento da cache

- **Como determinar se a informação está em cache?**
- **Em que posição da cache guardar os dados de uma posição de memória?**
- A memória cache tem uma capacidade pequena.
 - Exemplo: cache com 8 KB e memória externa com 1 MB
 - A memória externa é 128 vezes maior que a memória cache
- **O que fazer quando a posição de memória cache já está usada?**
 - Em geral, guardar a informação usada mais recentemente!
→ princípio da localidade

Cache de mapeamento direto: conceito



- Índice = (endereço em memória) mod (nº de blocos)
- Se (nº de blocos) = 2^b , a posição (i.e., o índice) é dada pelos **b** bits menos significativos do endereço.

Dados para este exemplo:

- Tamanho de bloco: 1 byte
- Cache (total): 8 bytes
- Memória principal: 32 bytes
- Índice = Endereço mod 8
(resto da divisão do endereço por 8)

Cache de mapeamento direto: etiquetas (tags)

- Mais que um endereço de memória é mapeado no mesmo bloco de cache.
 - Por exemplo: endereços 1, 9, 17 e 25 são mapeados no bloco 1.
- Dado um bloco de cache, como determinar o endereço de origem?

Solução

- Guardar o endereço associado aos dados guardados em cada bloco.
- Não é preciso guardar o endereço completo!
 - Basta guardar o quociente da divisão do endereço pelo número de blocos

Como se calcula rapidamente o quociente da divisão por uma potência de 2?

Exemplo (1/3)

Sequência de acessos: **10110₂ (22)**; 11010₂ (26); 10000₂ (16); 00011₂ (3); 10010₂ (18)

índice	val.	etiqueta	dados
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	0		
111	0		

Estado inicial: cache vazia

índice	val.	etiqueta	dados
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	1	10	MEM[22]
111	0		

Após acesso a 10**110**₂ (falta — *miss*)

Exemplo (2/3)

Sequência de acessos: 10110_2 (22); **11010_2 (26)**; **10000_2 (16)**; 00011_2 (3); 10010_2 (18)

índice	val.	etiqueta	dados
000	0		
001	0		
010	1	11	MEM[26]
011	0		
100	0		
101	0		
110	1	10	MEM[22]
111	0		

Após acesso a $11\textcolor{red}{0}10_2$ (falta —miss)

índice	val.	etiqueta	dados
000	1	10	MEM[16]
001	0		
010	1	11	MEM[26]
011	0		
100	0		
101	0		
110	1	10	MEM[22]
111	0		

Após acesso a $10\textcolor{red}{0}00_2$ (falta —miss)

Exemplo (3/3)

Sequência de acessos: 10110_2 (22); 11010_2 (26); 10000_2 (16); **00011_2 (3)**; **10010_2 (18)**

índice	val.	etiqueta	dados
000	1	10	MEM[16]
001	0		
010	1	11	MEM[26]
011	1	00	MEM[3]
100	0		
101	0		
110	1	10	MEM[22]
111	0		

Após acesso a $00\mathbf{011}_2$ (falta —miss)

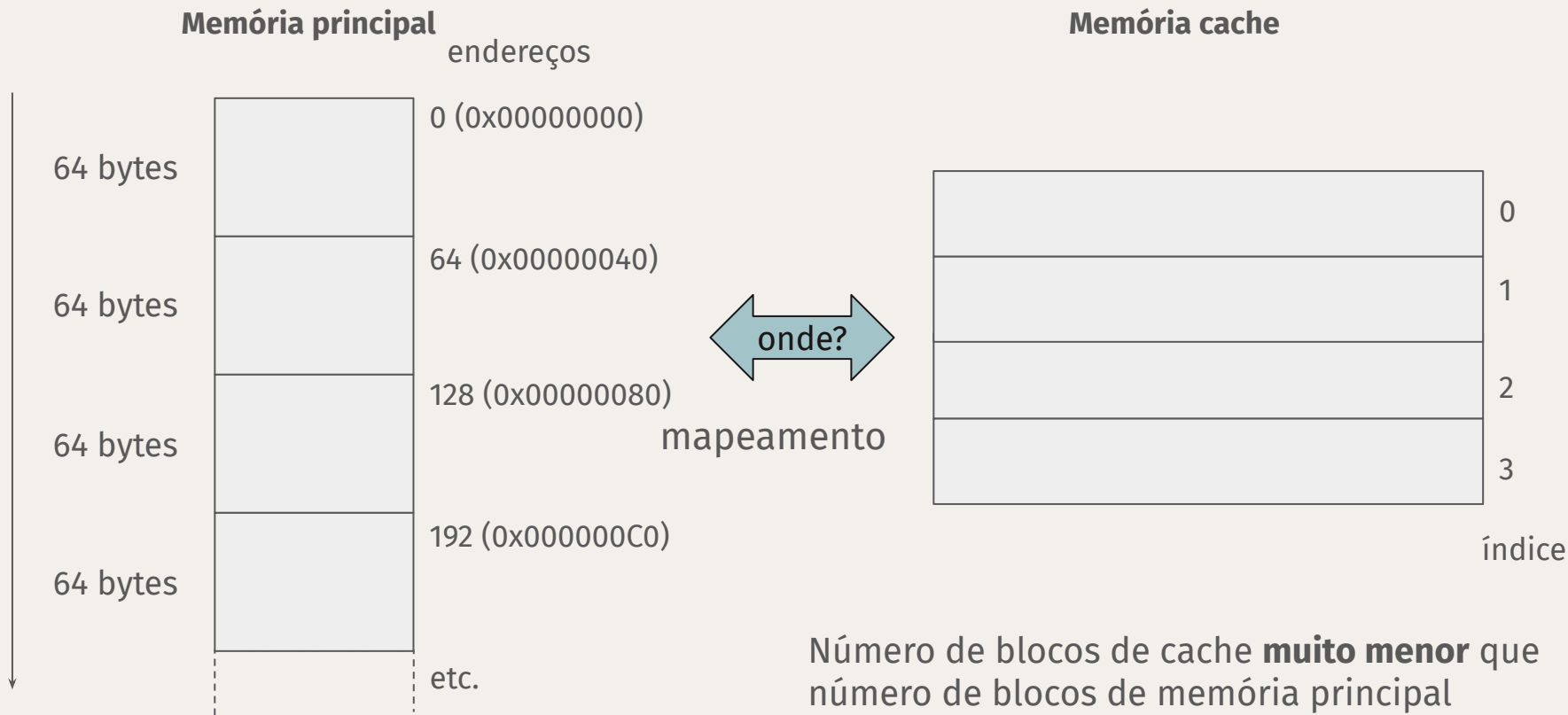
índice	val.	etiqueta	dados
000	1	10	MEM[16]
001	0		
010	1	10	MEM[18]
011	1	00	MEM[3]
100	0		
101	0		
110	1	10	MEM[22]
111	0		

Após acesso a $10\mathbf{011}_2$ (falta —miss), posição ocupada!

Tamanho de bloco da memória cache

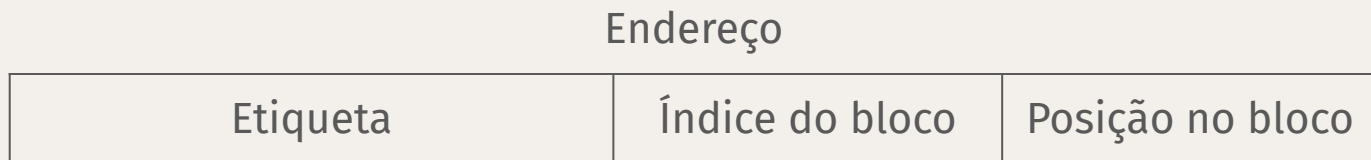
- Memória principal organizada conceitualmente em bytes
 - Os endereços (de dados ou instruções) são especificados em bytes.
- Tamanho de dados mais frequente: 32 bits ou 64 bits.
 - Na arquitetura RISC-V RV32, o item mais usado é a “palavra” (*word*) com 32 bits
 - O endereço de itens com mais de um byte é o endereço do seu primeiro byte em memória.
- Blocos (ou linhas) de cache contêm várias palavras consecutivas
 - O tamanho do bloco de cache é fixo em cada modelo.
 - Exemplo comum: 64 bytes/bloco (16 palavras por bloco)

Relação entre memória principal e memória de cache



Determinação de presença: etiquetas e bits de validade

- A etiqueta é constituída pelos bits mais significativos do endereço (bits não usados na determinação da posição em cache).
- Decomposição geral de um endereço:

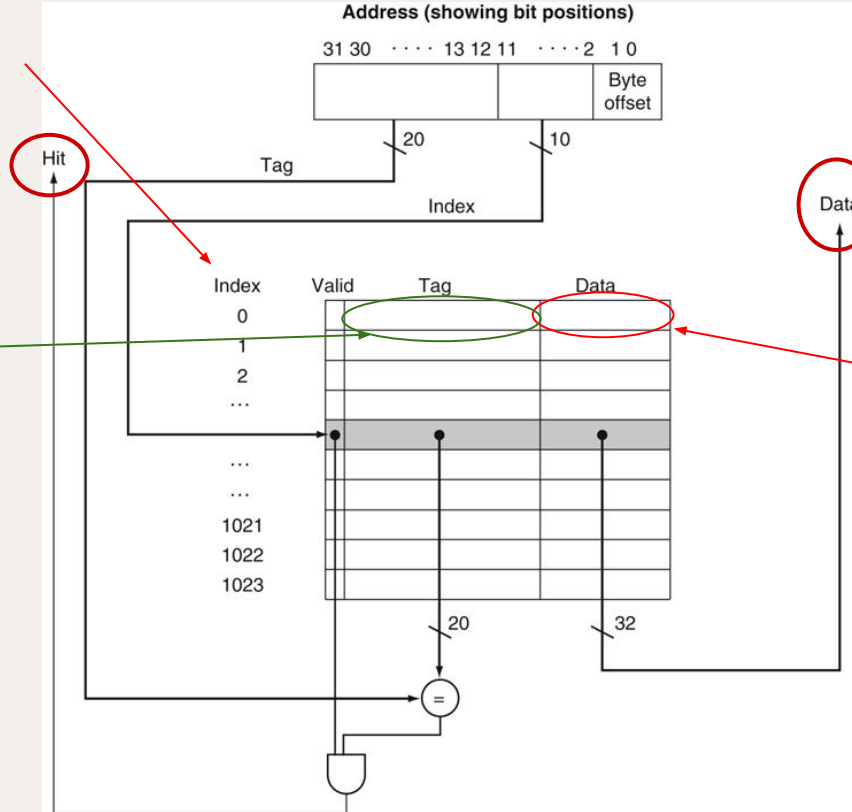


- Em certas situações, não existem conteúdos válidos na memória cache. Exemplo: arranque de um programa.
- Para detetar esta situação, cada bloco tem associado um **bit de validade**. Se bit = 0, então o bloco não tem conteúdo válido (a posição da cache está “vazia”).

Decomposição dos endereços para bloco de 4 bytes

índice
(nº do bloco na cache)

etiqueta

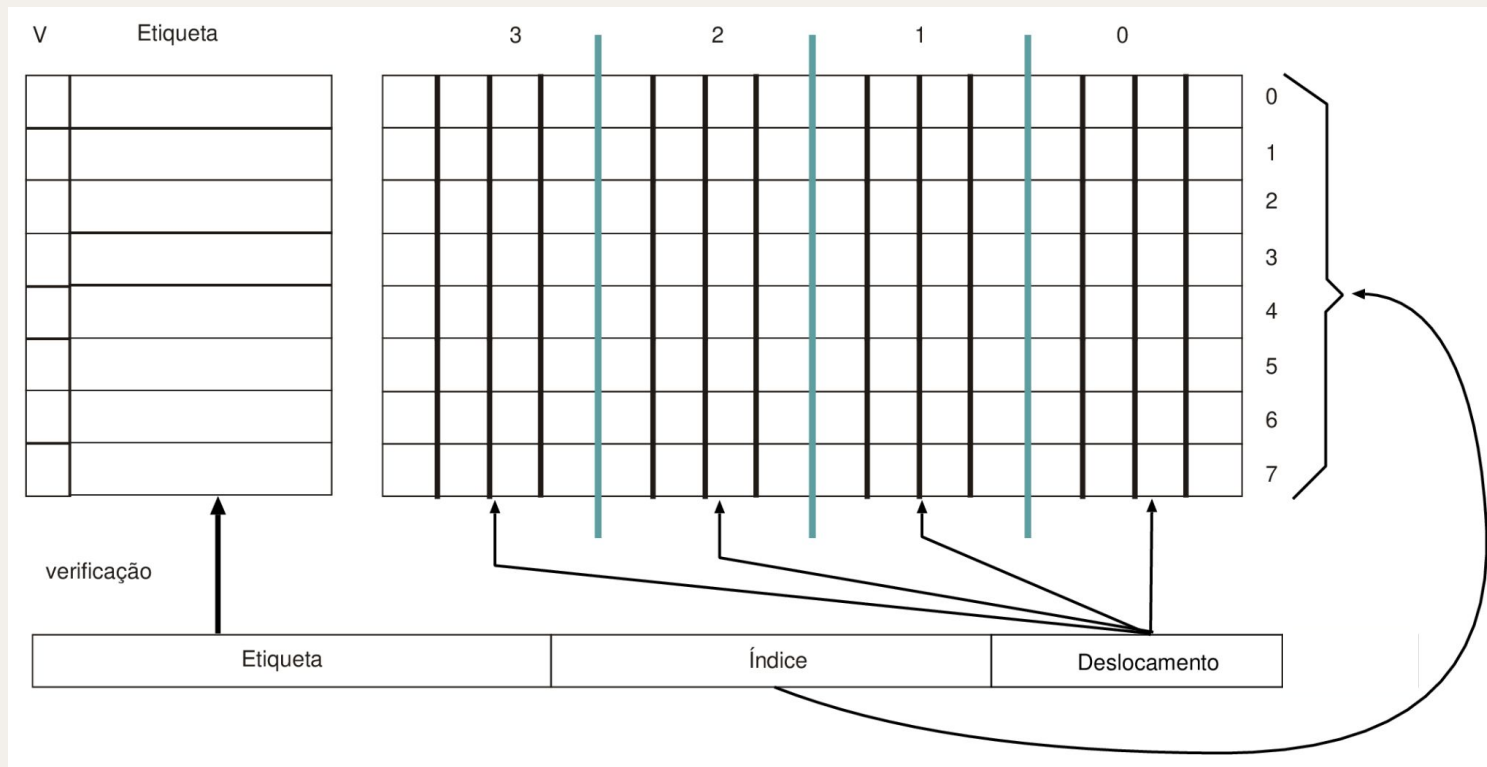


4 bytes
(1 palavra)

Blocos com várias palavras

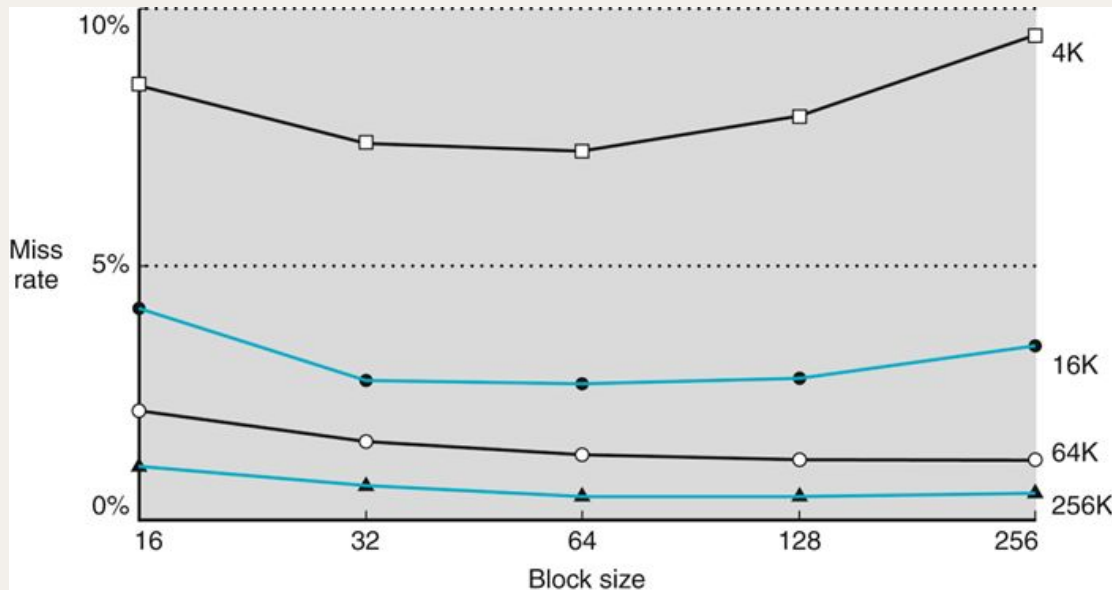
- O uso de uma palavra por bloco não aproveita a localidade espacial.
→ Devem ser usados blocos maiores, com L bytes ($L/4$ palavras).
- Para ser fácil de identificar elementos no interior do bloco, o número L deve ser uma potência de 2: $L = 2^w$ (o número de palavras é então 2^{w-2})
Cada byte pode ser identificado por w bits do endereço.
A parte do endereço que especifica a posição dentro do bloco designa-se por “deslocamento” (*offset*).
- Decomposição de um endereço de N bits:
 - Deslocamento: nº de bits = w
 - Índice: nº de bits = b
 - Etiqueta: nº de bits = $N-b-w$
- Frequentemente, os acessos a cache são feitos por palavra.
 - Endereços de palavras terminam com 2 zeros: $XX...XXXX00_2$ (alinhamento)

Estrutura de cache com múltiplas palavras por bloco



Cache com $b=3$ e $w=4$

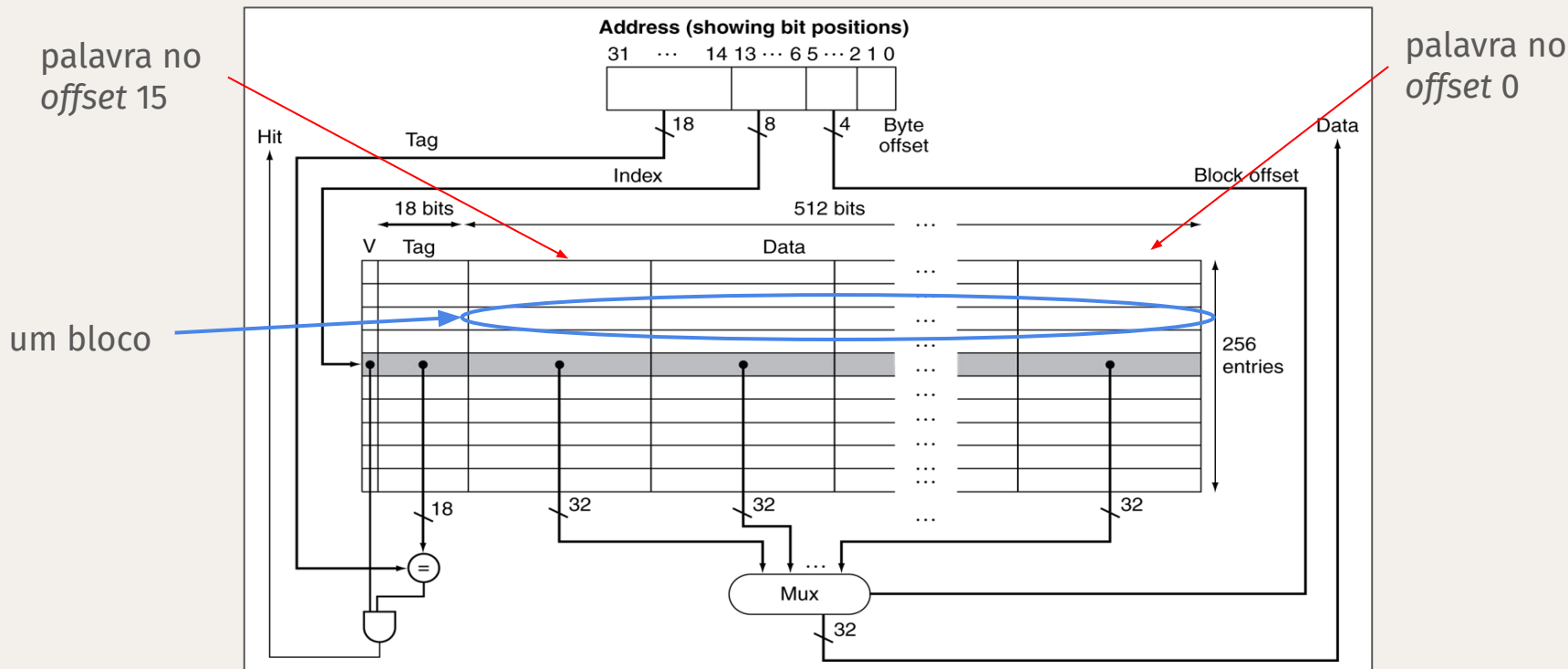
Tamanho de bloco vs. taxa de faltas



Para diferentes capacidades totais da memória cache

- Para tamanhos de blocos crescentes, a taxa de faltas diminui.
- Mas para tamanhos muito grandes, a taxa de faltas aumenta outra vez!
- A penalidade de falta tende a aumentar com a dimensão do bloco, porque é preciso ir buscar mais dados ao nível seguinte.

Decomposição de endereços para bloco de 64 bytes (1/2)

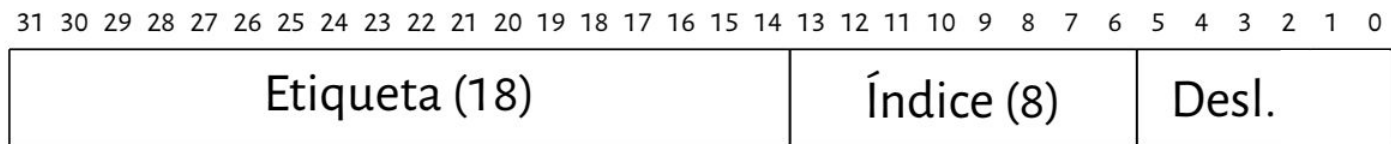


Cada bloco tem 16 palavras de 32 bits (4 bytes):

2 bits de *offset* de byte na palavra, 4 bits de *offset* da palavra no bloco

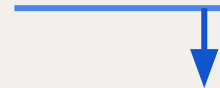
Decomposição de endereços para bloco de 64 bytes (1/2)

- Capacidade: 16 KiB, 256 blocos de 16 palavras.
- Etiqueta: 18 bits.
- Índice: 8 bits ($2^8 = 256$).
- Deslocamento (no bloco): 6 bits $2^6 = 64$.
- Exemplo de cálculo de posição:

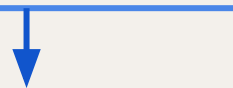


Endereço = 0×00457544 ($= 4552004_{10}$)

0000 0000 0100 0101 01 | 11 0101 01 | 00 0100



Etiqueta = 0×00115 ;

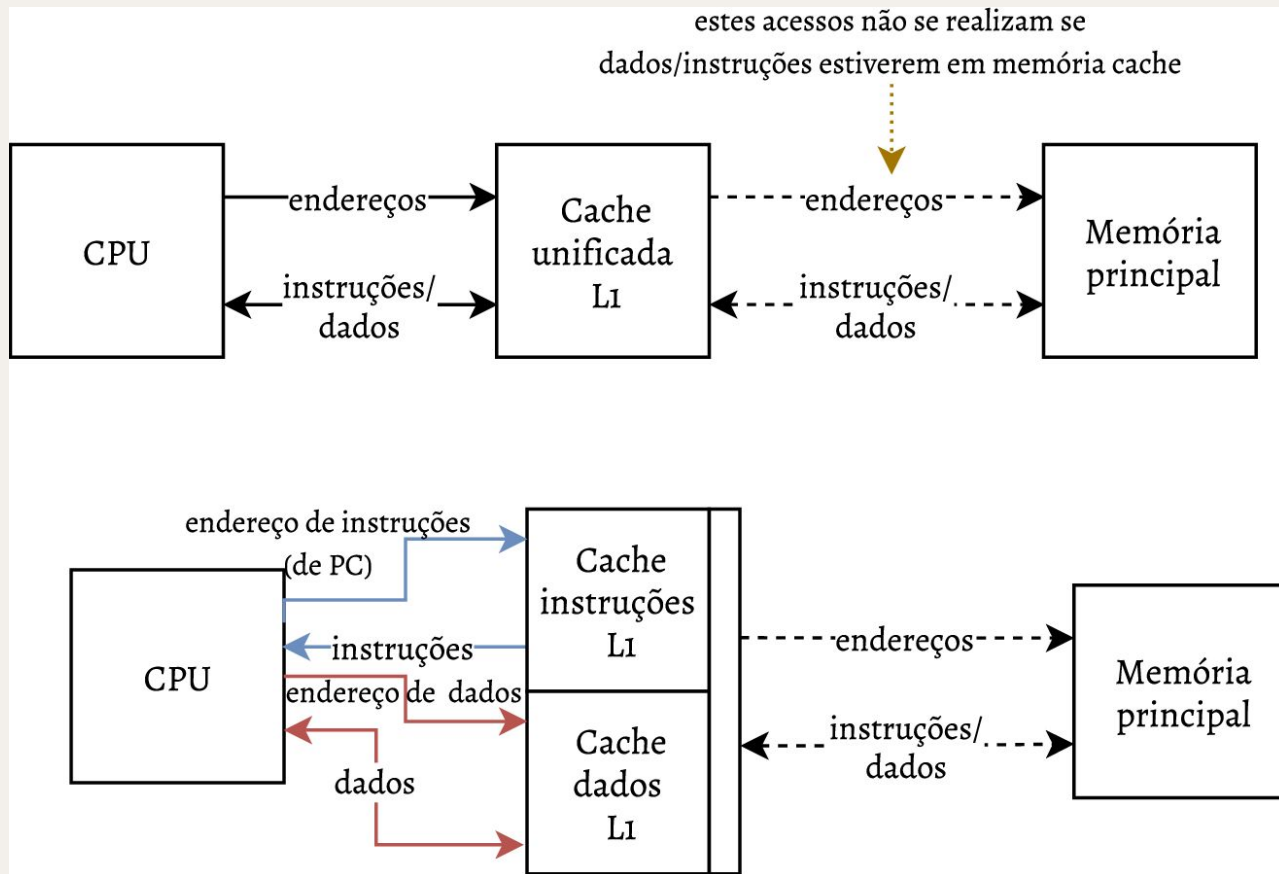


Bloco = $0 \times D5$ (213_{10})

Cache unificada vs. caches separadas (*split*)

- CPU faz dois tipos de acessos de leitura:
 - a. Obtenção de instruções
 - b. Obtenção de dados
- É possível (e benéfico) usar **duas memórias cache separadas** para cada tipo
 - I-cache: memória cache para instruções (apenas leitura)
 - D-cache: memória cache para dados (leitura e escrita)
- O uso de caches separadas evita conflitos estruturais no acesso à memória
 - É possível ler uma instrução e executar load/store no mesmo ciclo.
 - Esta é a organização usada no CPU de ciclo único estudado em FSC.
 - As memórias cache de nível 1 têm tipicamente esta organização.
 - Para uma mesma capacidade total, memórias cache unificadas (U-cache) permitem obter menores taxas de faltas (mas a diferença é pequena).

Memórias *cache* unificadas ou separadas



Tratamento de faltas

Faltas no acesso a memória *cache* (leitura)

- Em caso de acerto (*hit*) de *cache*, o CPU prossegue normalmente com os dados/instruções recebidos da memória *cache*.
- Em caso de falta (*cache miss*) na leitura:
 - Obter bloco a partir do próximo nível de hierarquia
 - Falta na *cache* de instruções:
 - Reiniciar a obtenção da instrução
 - Falta na *cache* de dados:
 - Completar acesso aos dados

Acerto de escrita: *write-through* ou *write-back*

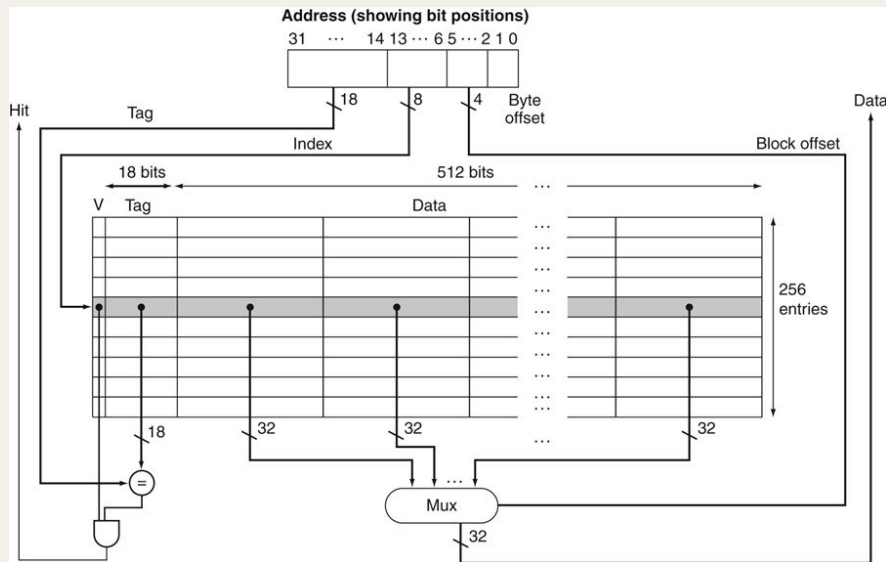
- Como tratar uma operação de escrita num endereço cujos dados estão presentes em *cache* (*write hit*) ?
 - Alternativa 1: alterar (atualizar) *cache* e memória principal: ***write-through***
 - *Cache* e memória ficam com os mesmos valores (consistentes).
 - Instruções *store* continuam a ter grande impacto, porque implicam sempre um acesso à memória principal.
 - Alternativa 2: alterar **apenas** a memória *cache*: ***write-back***
 - Usar **1 bit por bloco** para indicar se o bloco foi alterado ou não.
 - Quando a posição em *cache* de um bloco alterado é reutilizada para outro bloco:
 - Gravar conteúdo do bloco de *cache* em memória principal (para garantir consistência dos dados) antes de reutilizar o bloco.
 - Quando o programa termina, é necessário escrever eventuais blocos alterados para memória principal (*cache flush*).

Falta de escrita (*write miss*)

- Como tratar a escrita para um endereço cujo conteúdo não está na memória *cache*?
- Alternativa 1: atualizar diretamente a memória principal (***no-write-allocate***)
 - O conteúdo da memória *cache* não é alterado.
 - Eficaz quando os programas inicializam blocos de memória antes de a usarem.
- Alternativa 2: obter o bloco de memória (***allocate-on-miss***)
 - Atualizar memória *cache*.
- Logicamente, o tratamento de faltas de escrita é independente do tratamento dos acertos de escrita, mas tipicamente:
 - Política de *write-through* é usada com *no-allocate* (aposta na simplicidade).
 - Política de *write-back* é usada com *allocate-on-miss* (reduz acessos à memória principal).

Exemplo: Instrinsity FastMATH

- Memória *cache* usada num processador MIPS
- *Caches* separadas
 - Cada uma com 16 KB: 256 blocos de 16 palavras
 - D-cache: *write-through* ou *write-back*
- Taxas de faltas para SPEC2000
 - I-cache: 0,4%
 - D-cache: 11,4%
 - Média pesada: 3,2%

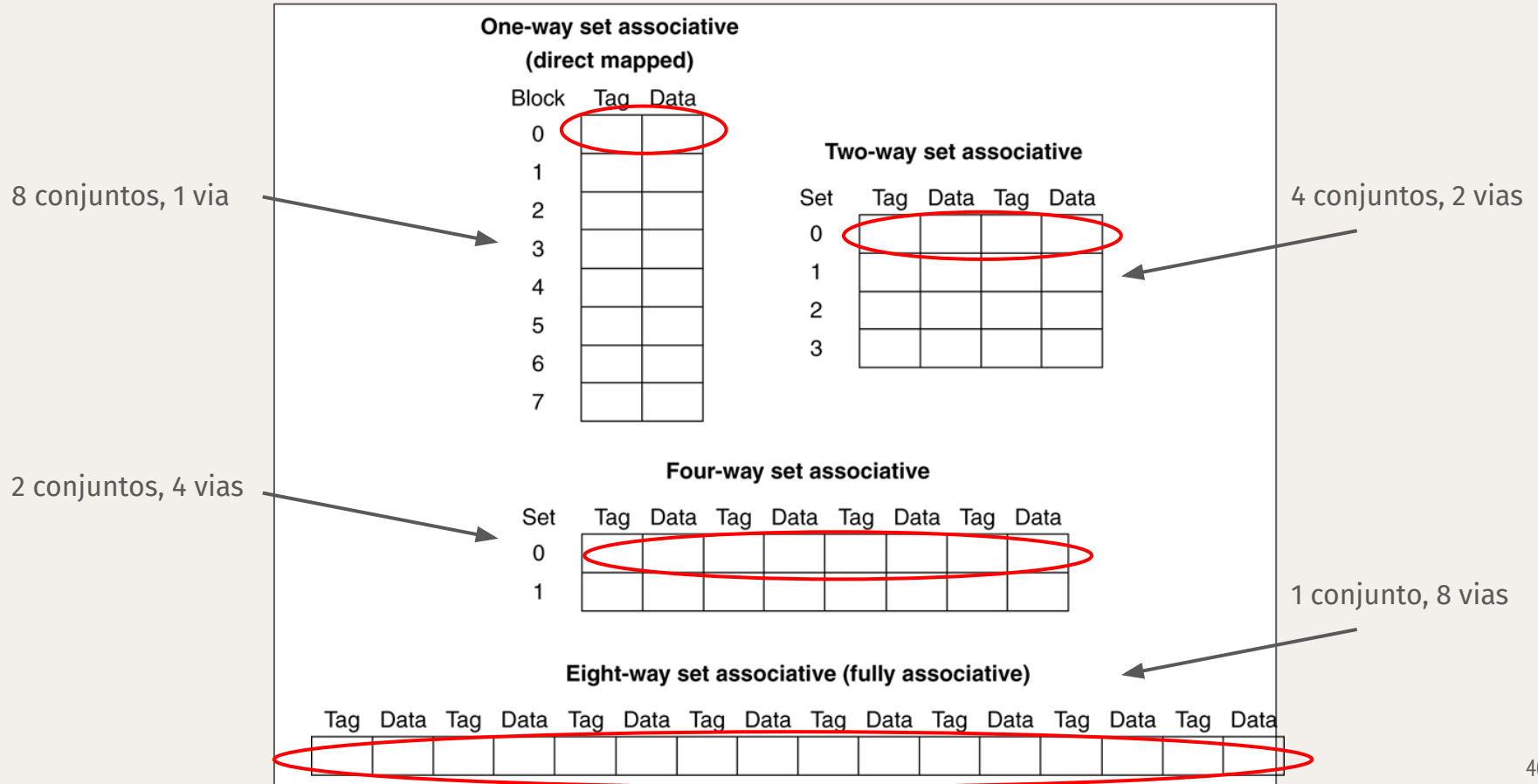


Redução da taxa de faltas

Associatividade

- A taxa de faltas é influenciada pela flexibilidade na colocação de blocos de memória na *cache*.
 - Mapeamento direto: um endereço $\rightarrow$ uma posição de *cache* (índice)
- Memória *cache* completamente associativa (*fully associative*)
 - Bloco de memória pode ser colocado em qualquer posição (índice) da *cache*.
 - É preciso comparar todas as etiquetas simultaneamente para determinar a presença de informação em *cache*
 - Requer muitos circuitos lógicos (um comparador por bloco)
- Memória *cache* associativa por conjuntos (*n-way set associative*)
 - Cada conjunto tem **n** blocos, um por cada via
 - **Índice do bloco** determina o conjunto.
 - Mas não qual dos blocos do conjunto
 - É preciso comparar simultaneamente todas as etiquetas **de um conjunto**
 - Requer **n** comparadores de etiquetas

Grau de associatividade: *cache* de 8 blocos



Exemplos do efeito da associatividade (1/2)

- Comparar organização de memória *cache* com 4 blocos
 - Mapeamento direto
 - Associativa com conjuntos de 2 blocos (2-way)
 - Completamente associativa
- Sequência de acesso a blocos: 0, 8, 0, 6, 8
- Mapeamento direto:

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

Exemplos do efeito da associatividade (2/2)

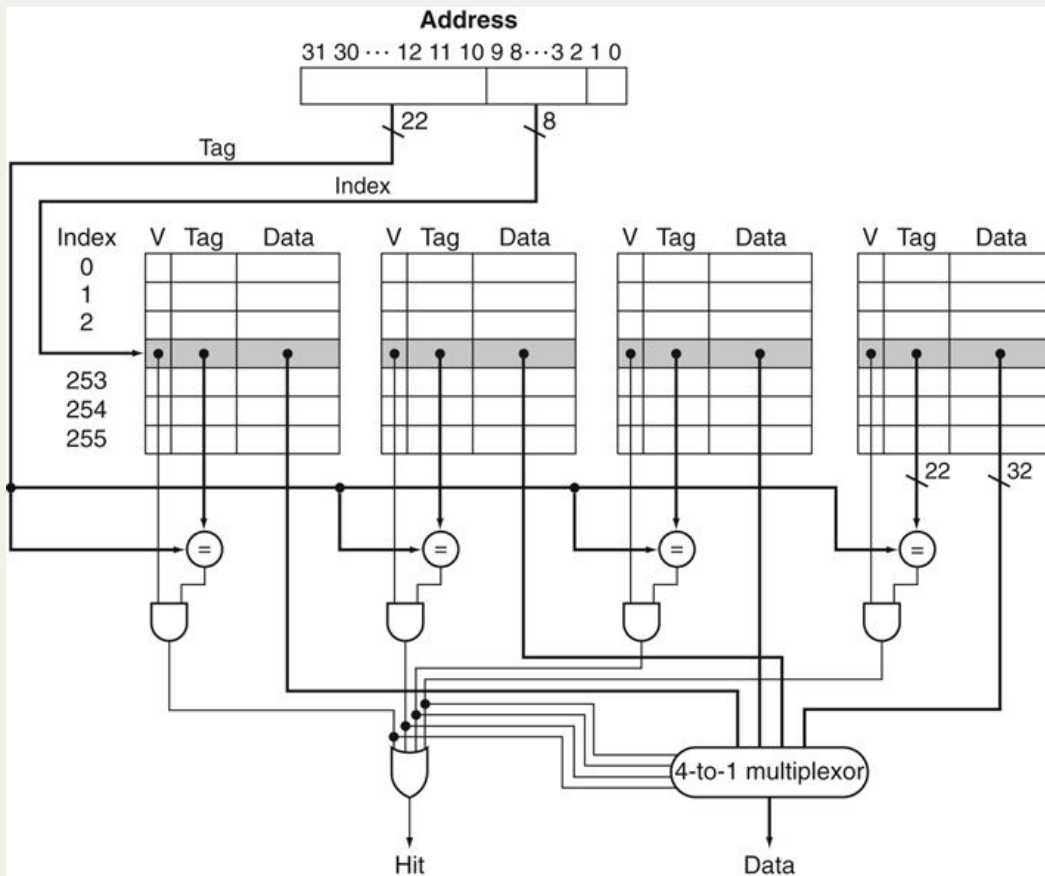
- Associativa com conjuntos de 2 blocos (2-way)

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	Mem[0]			
8	0	miss	Mem[0]	Mem[8]		
0	0	hit	Mem[0]	Mem[8]		
6	0	miss	Mem[0]	Mem[6]		
8	0	miss	Mem[8]	Mem[6]		

- Completamente associativa

Block address		Hit/miss	Cache content after access			
0		miss	Mem[0]			
8		miss	Mem[0]	Mem[8]		
0		hit	Mem[0]	Mem[8]		
6		miss	Mem[0]	Mem[8]	Mem[6]	
8		hit	Mem[0]	Mem[8]	Mem[6]	

Organização de *cache* associativa (4-way)

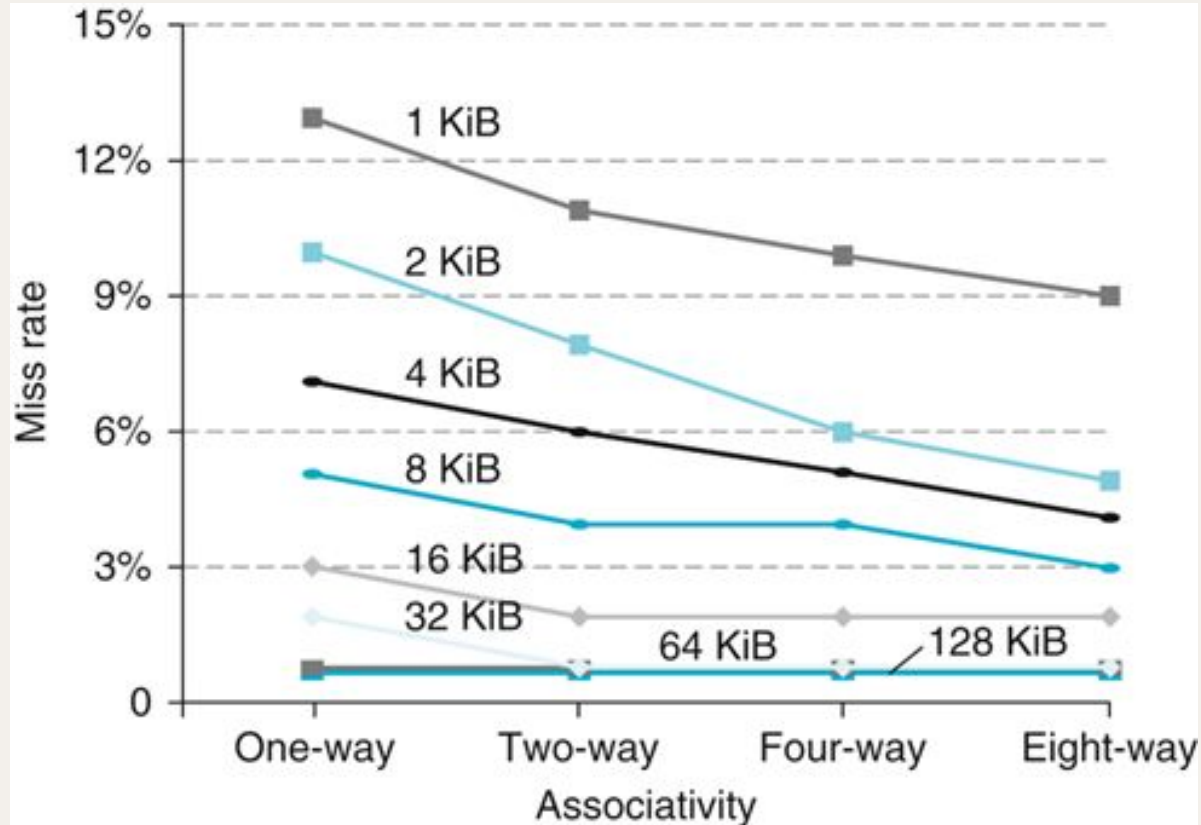


Associatividade	Taxa de faltas (D-cache)
1	10,3%
2	8,6%
4	8,3%
8	8,1%

Política de substituição

- Associatividade requer uma política de substituição
 - **Que bloco de um conjunto usar?**
 - Preferir bloco inválido (livre), se existir algum
- Mapeamento direto ($n=1$): sem escolha
- Associatividade por conjuntos
 - **Política LRU** (*least-recently used*)
 - Escolher o bloco não acedido há mais tempo
 - Simples $n=2$, viável para $n=4$, demasiado complicado para $n > 4$
 - **Política de seleção aleatória** (“à sorte”)
 - Se n for elevado, os resultados são semelhantes aos de LRU.
 - **Política FIFO** (first-in, first-out)
 - Existem outras ...

Taxas de faltas (D-Cache) vs. associatividade



Múltiplos níveis de memória *cache*

- Como reduzir a penalidade de falta?
 - Reduzir o tempo de acesso ao nível de memória seguinte!
- Memória *cache* primária ligada ao CPU (nível L1)
 - Pequena, mas rápida; tipicamente, *caches* separadas
- Memória de *cache* de nível 2 (L2) para acessos que causam faltas na *cache* L1.
 - Memória *cache* maior e mais lenta que a *cache* L1
 - Ainda assim, mais rápida que a memória principal
 - Tipicamente, é uma *cache* unificada
- Faltas na *cache* L2 resultam em acessos à memória principal
- Alguns sistemas topo de gama incluem memórias *cache* L3.

Parâmetros de *cache* típicos

Feature	Typical values for L1 caches	Typical values for L2 caches
Total size in blocks	250–2000	2500–25,000
Total size in kilobytes	16–64	125–2000
Block size in bytes	16–64	64–128
Miss penalty in clocks	10–25	100–1000
Miss rates (global for L2)	2%–5%	0.1%–2%

Exemplos comerciais de caches multi-nível

Characteristic	ARM Cortex-A53	Intel Core i7
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	Configurable 16 to 64 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	Two-way (I), four-way (D) set associative	Four-way (I), eight-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, variable allocation policies (default is Write-allocate)	Write-back, No-write-allocate
L1 hit time (load-use)	Two clock cycles	Four clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 2 MiB	256 KiB (0.25 MiB)
L2 cache associativity	16-way set associative	8-way set associative
L2 replacement	Approximated LRU	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate	Write-back, Write-allocate
L2 hit time	12 clock cycles	10 clock cycles

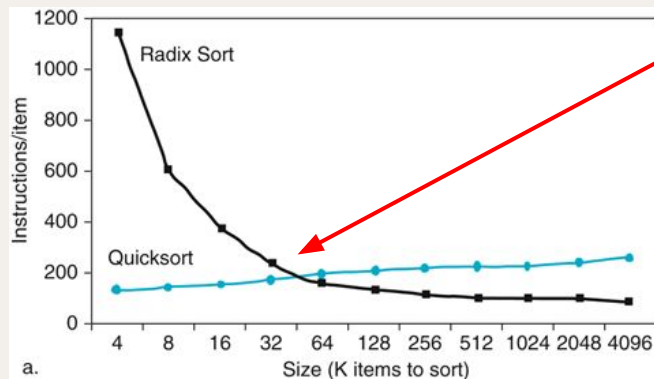
Characteristic	ARM Cortex-A53	Intel Core i7
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement	–	Approximated LRU
L3 block size	–	64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles

Hierarquia de memória: resumo dos conceitos básicos

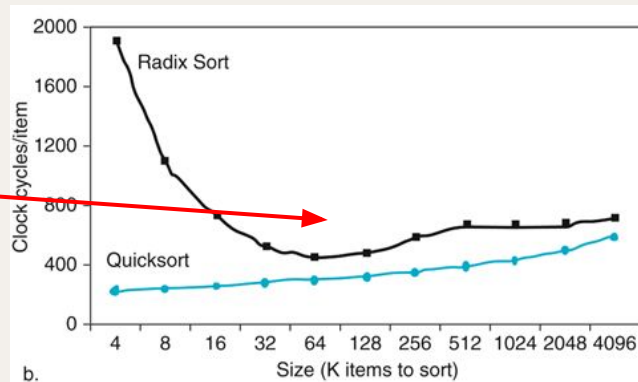
- O nível **n** é mais rápido e tem menor capacidade que o nível **n+1**.
- **Bloco**: quantidade mínima de informação num determinado nível.
A informação entre níveis é transferida em blocos.
- **Acerto** (no nível **n**): acesso ao nível **n** encontrou informação pretendida (*hit*).
- **Falta**: acesso ao nível **n** não encontrou a informação (*miss*).
- **Taxa de acertos**: $(\text{nº de acertos})/(\text{nº de acessos})$ [*hit rate*]
- **Taxa de faltas**: $(\text{nº de faltas})/(\text{nº de acessos}) = 1 - (\text{taxa de acertos})$ [*miss rate*]
- **Tempo de acerto**: tempo de acesso ao nível **n**, incluindo a verificação da presença do item pretendido (*hit time*).
- **Penalidade de falta**: tempo necessário para copiar o item pretendido do nível **n+1** para o nível **n**, mais o tempo necessário para o nível **n** terminar o atendimento do acesso original (*miss penalty*).

Desempenho com memórias *cache*

Interação entre memória *cache* e programas

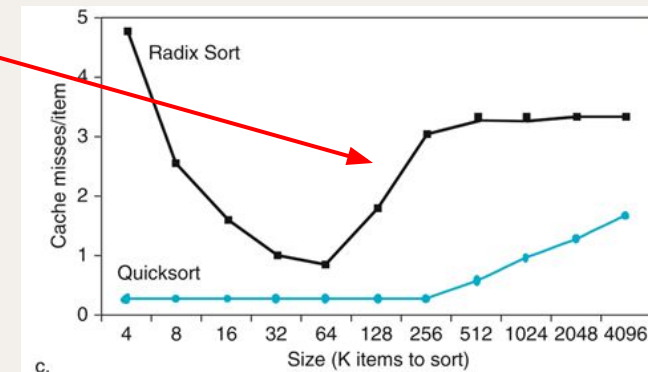


Radix sort executa menos instruções que Quicksort, mas Quicksort é mais rápido!



Porquê?

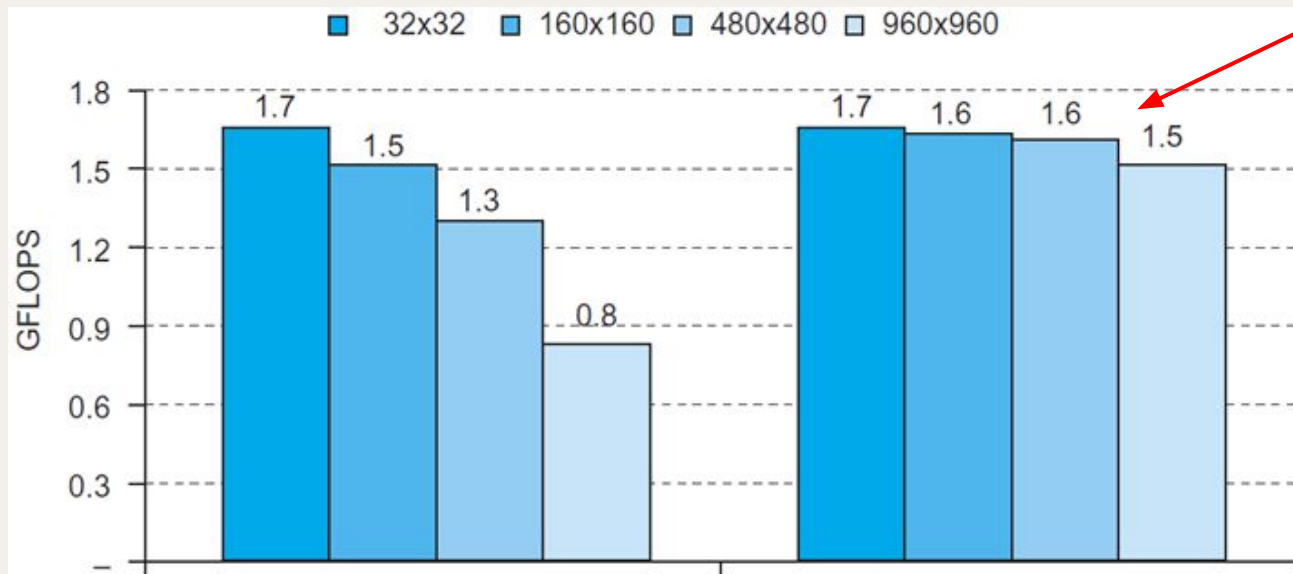
Menor taxa de faltas.
Desempenho depende do padrão de acessos à memória.



- Memórias *cache* têm um impacto muito importante no desempenho de sistemas
- Os efeitos são muito influenciados pela taxa de acertos em combinação com o tempo de acerto (*hit time*)

Exemplo: operação sobre matrizes

- *Benchmark* DGEMM: operação de matrizes $C = A \times B + C$



Versão otimizada atinge bom desempenho para todos os tamanhos de matrizes (mais de 1,5 GFLOPS)

GFLOPS: Giga *floating point operations* (10^9) per second

Modelo de desempenho com memórias *cache*

- Componentes do tempo de CPU
 - Ciclos de relógio usados em execução
 - Incluem o tempo de aceder a informação em *cache* L1
 - Ciclos de protelamento no acesso à memória (C_{pmem})
 - Principalmente causados por faltas
- Análise simplificada:

$$C_{\text{pmem}} = \# \text{ acessos a memória} \times \text{taxa de faltas} \times \text{penalidade de falta}$$

$$= \# \text{ instruções} \times \frac{\# \text{ faltas}}{\# \text{ instruções}} \times \text{penalidade de falta}$$

A penalidade de falta é geralmente dada em número de ciclos de relógio.

Exemplo de cálculo de desempenho

- Dados:
 - Taxa de faltas de I-cache: 2%
 - Taxa de faltas de D-cache: 4%
 - Penalidade de falta: 100 ciclos
 - CPI ideal (de base) = 2
 - Instruções de *load/store* constituem 36% das instruções
- Ciclos de protelamento por instrução
 - I-cache: $0,02 \times 100 = 2$
 - D-cache: $0,36 \times 0,04 \times 100 = 1,44$
- CPI efetivo = $2 + 2 + 1,44 = 5,44$ (valor a usar na equação fundamental)
 - O CPU ideal seria $5,44 / 2 = 2,72$ vezes mais rápido

Tempo médio de acesso

- O tempo de acerto também é relevante para o desempenho
- Tempo médio de acesso à memória (TMAM)
 - $TMAM = \text{tempo de acerto} + \text{taxa de faltas} \times \text{penalidade de falta}$
- Exemplo:
 - Período do relógio: 1 ns
 - Tempo de acerto: 1 ciclo
 - Penalidade de falta: 20 ciclos
 - taxa de faltas (I-Cache): 5%
- $TMAM = 1 + 0,05 \times 20 = 2 \text{ ns}$
 - Tempo médio de acesso por instrução: 2 ciclos

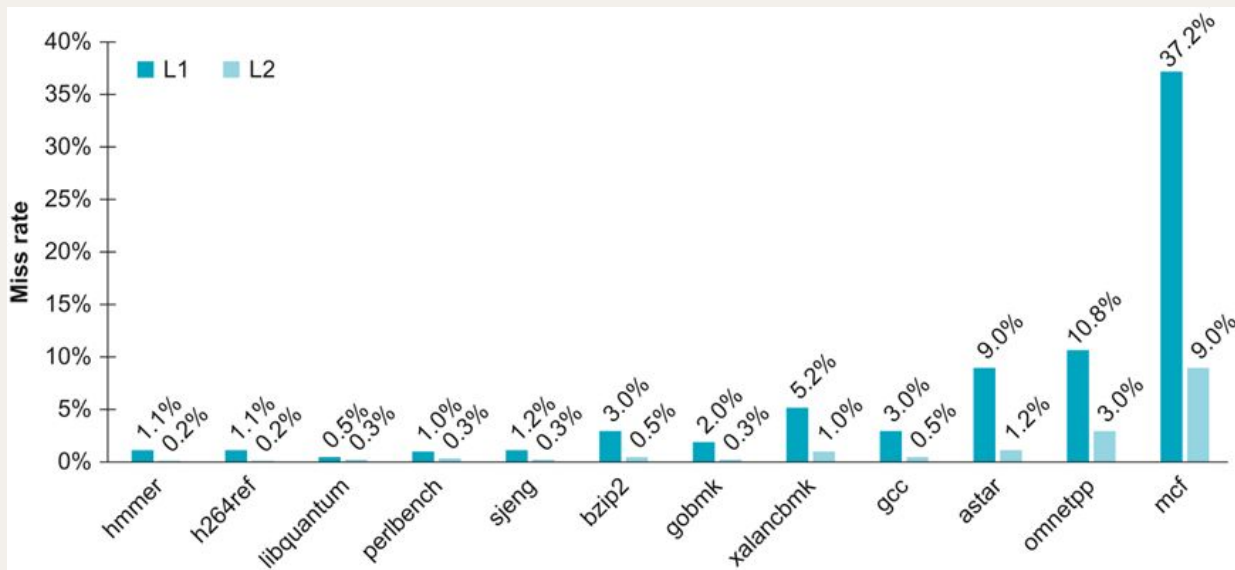
Desempenho com 2 níveis de *cache*

- Dados:
 - CPU de base CPI = 1, frequência de relógio = 4 GHz
 - Número de faltas/instrução = 2%
 - Tempo de acesso à memória principal = 100ns
- Apenas com *cache* L1
 - Penalidade de falta = $100 \text{ ns} / 0,25 \text{ ns} = 400$ ciclos
 - CPI efetivo = $1 + 0,02 \times 400 = 9$
- Com *cache* L2
 - Tempo de acesso: 5 ns
 - Taxa de faltas global (para memória principal) = 0,5 %
- Falta em L1 e acerto em L2
 - Penalidade = $5 \text{ ns} / 0,25 = 20$ ciclos
- Falta em L1 e em L2
 - Penalidade extra: 400 ciclos
- CPI efetivo = $1 + 0,02 \times 20 + 0,005 \times 400 = 3,4$

Desempenho aumenta:
 $9/3,4 = 2,6\times$

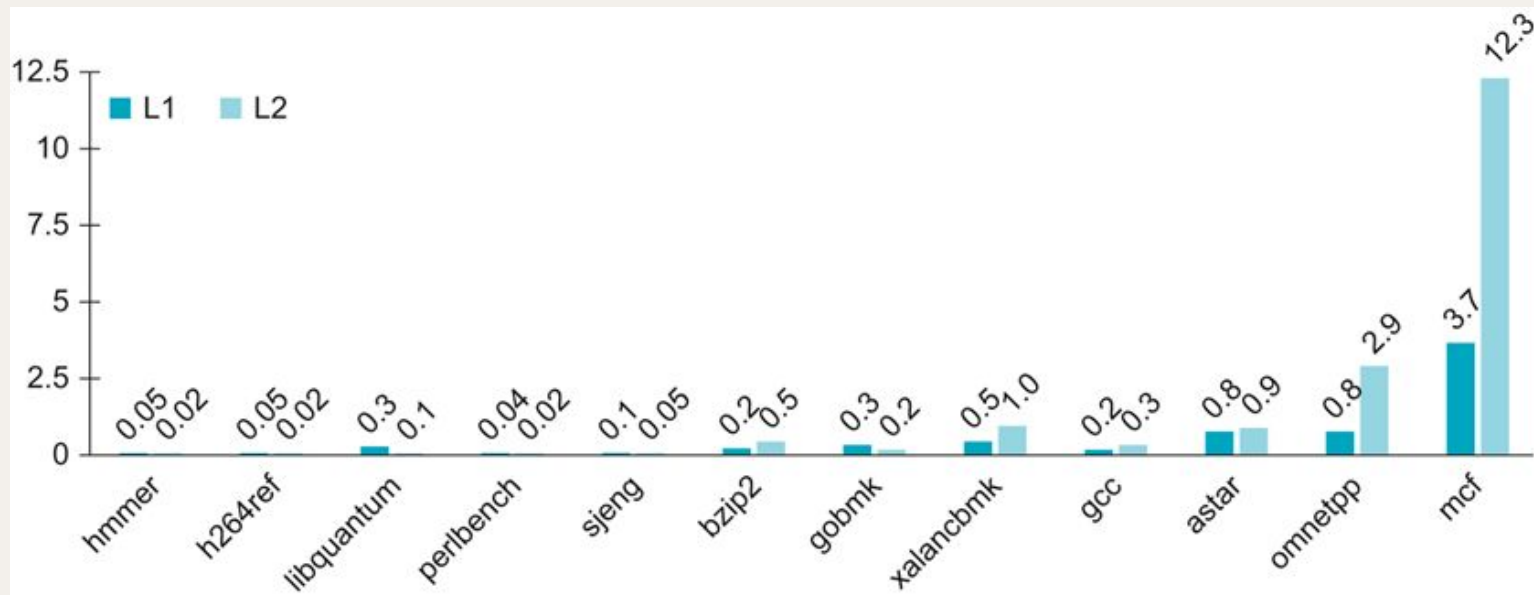
Dados empíricos: taxa de faltas

- ARM A53 com 32 KiB L1 e 1 MiB L2 (SPECInt2006)
 - Taxa de L2 é global (inclui todos os acessos).
 - Aplicações têm comportamentos muito diferentes.



Dados empíricos: penalidade por acesso à memória

Como a penalidade de falta em L2 é grande, a penalidade é elevada, mesmo para taxas de faltas baixas.



Resumo sobre desempenho

- Quando o desempenho do CPU aumenta:
 - Penalidade de faltas tem impacto significativo
- Quando o CPI ideal baixa:
 - A proporção de tempo despendido em protelamentos de memória aumenta
- Quando a frequência aumenta:
 - Protelamentos de memória gastam mais ciclos de CPU
- A memória *cache* não pode ser negligenciada no cálculo do desempenho.

Bibliografia

- Secções 5.1-5.4, 5.13, de David A. Patterson & John L. Hennessy
“Computer Organization and Design” RISC-V edition 2nd ed. Elsevier.